

Beyond Strike Rate

A hierarchical Bayesian model for T20 batting ability across four leagues

Group project · Bayesian Statistics · IPM Term VII · Prof. Sayantan Banerjee · IIM Indore
An interactive companion runs the model, the sampler and the shortlist in the browser.

Summary. A T20 strike rate is an average over a small number of balls, and such averages are noisy. Counting runs off the bat across 221,164 deliveries shows that a 100-ball strike rate carries ± 16.7 points of sampling noise on its own, before any question about ability is raised. We fit a hierarchical Normal model to 839 player $\times$ league cells from the IPL, CPL, BBL and men's T20 internationals, with one offset per league so that a CPL strike rate can be read on the IPL scale. Each batter's ability is returned as a posterior distribution on that scale, so a shortlist becomes a probability statement rather than a ranking. On the 2025 IPL, held out for testing, the hierarchical model predicts better than the raw leaderboard for every group of batters, and for the eight batters with an overseas record but no IPL history it is the only method that gives an estimate.

1 The business problem

Problem Set 5 opens with a national health agency comparing 30-day mortality rates across 50 hospitals. A newspaper ranks the hospitals by their raw sample means and declares a small rural hospital with eight patients the best in the country. The ranking is not credible, because eight patients carry very little information about that hospital.

An IPL auction ranks batters in the same way. A batter records a strike rate of 234, and that number is used to price him. The top fifteen of the raw IPL leaderboard for 2021–2024 have a median of 175 balls behind their strike rates, and chance alone can move a strike rate over that many balls by about ± 13 points.

1.1 How much noise is in a strike rate?

Before any model, count. A ball off the bat produces 0, 1, 2, 3, 4, 5 or 6 runs, and in practice almost only 0, 1, 2, 4 and 6. Across the 221,164 deliveries in our window the mean is 1.320 runs per ball with variance 2.798. A strike rate multiplies by 100, so the per-ball variance on the strike-rate scale is $\sigma^2 \approx 27,978$, and the standard deviation of an observed strike rate over n balls is $\sigma/\sqrt{n}$. Putting $n = 100$:

A strike rate over 100 balls carries ± 16.7 points of sampling noise. This number requires no model. It follows from counting runs. Section 8 reports what the fitted model makes of it.

1.2 The decision rule

Following PS1 Q9(h)–(i) and PS1 Q11(i), inference and decision are kept apart. The model's job is to return a posterior for each batter's ability θ_i , expressed as an expected IPL strike rate. The franchise's job is to say what it wants and how sure it needs to be:

$$\text{shortlist batter } i \iff \Pr(\theta_i > c \mid \text{data}) > p,$$

where c is the strike rate the franchise is buying and p is the confidence it requires. Our baseline is $c = 140$, $p = 0.80$. This is the rule in PS1 Q11(i) — *design a play for Arjun only if $\Pr(\theta > 0.40 \mid X) > 0.80$* — applied to a different sport, with the probability estimated from MCMC output as in PS4 Q9(f).

Both c and p are business choices, not statistical ones. Section 10 reports what happens when a scout, an analyst and a finance head set them differently.

2 Data

2.1 The data

Cricsheet’s ball-by-ball archives for four competitions, 2021 to 2025: the IPL, the Caribbean Premier League, the Big Bash League and men’s T20 internationals. Deliveries are aggregated to **player $\times$ league cells**. For a cell we record n , the number of balls faced, and y , the strike rate over those balls. Wides are dropped, because a wide is not a ball the batter faced. No-balls are kept: the batter faces them and can score off them. Cells with fewer than 25 balls are dropped, which is what lets us treat y as an approximately Normal average.

Fitting uses 2021–2024. The 2025 IPL is held out entirely and never seen until Section 9.2.

League	Cells	Balls	Run rate	Median balls	Range of cell SR
IPL	194	63,658	139.5	158	43–234
CPL	143	28,497	128.3	118	45–166
BBL	191	48,620	130.2	133	61–187
T20I	311	80,389	130.4	142	33–200
All	839	221,164	132.7	138	33–234

Table 1. The four competitions after filtering. “Run rate” is the ball-weighted mean strike rate, which is what a scorecard would report.

2.2 Restricting T20 internationals to Full Member matches

Our first fit used every men’s T20 international in the archive. These matches involve **105 different national teams**, from India and Australia to Estonia, Mongolia and Bhutan. A single offset cannot describe a competition this heterogeneous: the model has no way of distinguishing a strike rate of 190 against Cyprus from one against Australia. It returned a shortlist headed by batters from associate nations who had recorded very high strike rates against much weaker bowling.

We therefore keep only T20Is in which *both* sides are ICC Full Members. The cost was small where it matters: the number of batters bridging the IPL and T20I fell from 107 to 106. The population mean moved from 121.6 to 132.2, and the shortlist became recognisable. Both shortlists are kept in the repository; the discarded one is `figs/F10_shortlist_ALLT20I_before.pdf`.

2.3 Bridge players and identification

	IPL	CPL	BBL	T20I
IPL	194	38	36	106
CPL	38	143	26	69
BBL	36	26	191	55
T20I	106	69	55	311

Table 2. Batters appearing in both competitions, 2021–2024. The diagonal is the number of batters in that competition. Off-diagonal entries are the only source of information about the league offsets.

106 batters link the IPL to T20 internationals, 38 to the CPL and 36 to the BBL. The IPL–T20I link is well supported. The CPL and BBL links are thinner and depend partly on the T20I route: a batter observed in both the CPL and T20Is, and another in both T20Is and the IPL, together connect the CPL to the IPL indirectly. The consequence appears in the width of the credible intervals in Section 8.

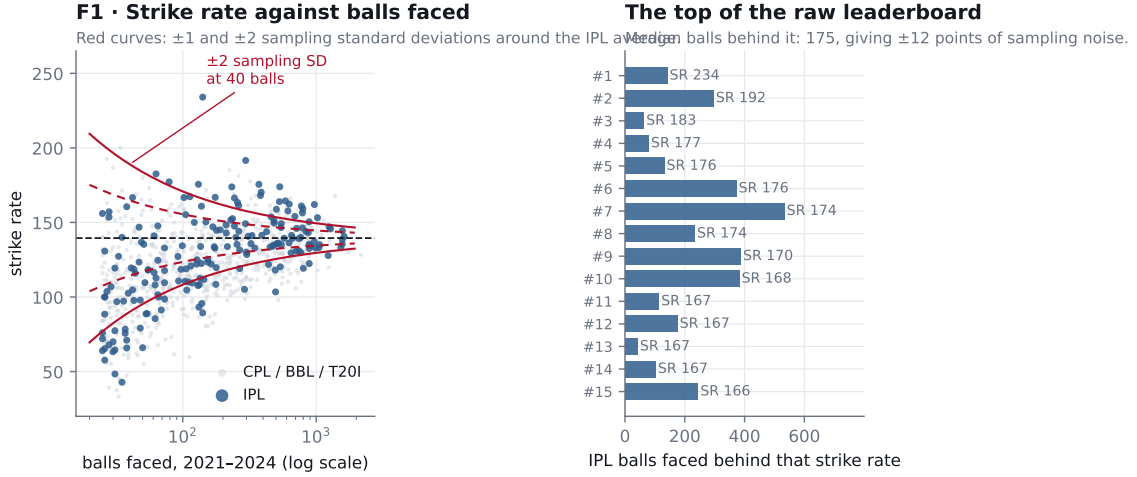


Figure 1. Left: every cell, against the number of balls behind it. The red curves are ± 1 and ± 2 sampling standard deviations around the IPL average — the spread chance alone would produce. Right: the top of the raw IPL leaderboard and the careers behind it.

Figure 1 shows the problem directly. Extreme strike rates almost all occur on the left of the plot, where few balls have been faced. On the right, where careers are long, the strike rates lie close to the average.

3 Model specification

We compare four models, following Sessions 17–18 and *Bayes Rules!* Chapter 16.

M0, complete pooling. A single θ for all batters. It cannot support a shortlist, but it is the baseline the other models are measured against.

M1, no pooling. Each batter has their own θ , estimated from their own record and nothing else. *This is the raw leaderboard written as a model.* It gives no estimate for a batter with no IPL record.

M2, partial pooling, IPL only. The hierarchical Normal model, verbatim:

$$y_i \mid \theta_i, \sigma^2 \sim N\left(\theta_i, \frac{\sigma^2}{n_i}\right), \quad \theta_i \mid \mu, \tau^2 \sim N(\mu, \tau^2),$$

with $\mu \sim N(m_0, s_0^2)$, $\tau^2 \sim \text{IG}(a_\tau, b_\tau)$ and $\sigma^2 \sim \text{IG}(a_\sigma, b_\sigma)$. The likelihood is written for the cell mean, so a long career enters with a small variance and a short one with a large variance. The term σ^2/n_i is how the model accounts for balls faced.

M3, partial pooling with league offsets. The model used for the results below:

$$y_{i\ell} \mid \theta, \delta, \sigma^2 \sim N\left(\theta_i + \delta_\ell, \frac{\sigma^2}{n_{i\ell}}\right), \quad \theta_i \sim N(\mu, \tau^2), \quad \delta_\ell \sim N(0, \omega^2), \quad \delta_{\text{IPL}} = 0.$$

3.1 Why we fix $\delta_{\text{IPL}} = 0$

Without a constraint the model cannot distinguish every batter being 5 points better from every league being 5 points easier: adding a constant to all θ and subtracting it from all δ leaves the likelihood unchanged. Fixing one league fixes the origin. We fix the IPL, and the benefit is one of interpretation:

With $\delta_{\text{IPL}} = 0$, θ_i means exactly “this batter’s expected strike rate in the IPL” — including for batters who have never played in the IPL. Their CPL, BBL and T20I strike rates are translated onto the IPL scale by the fitted offsets. This is what makes the model usable for the decision stated in Section 1.

F2 • Directed acyclic graph for M3

Arrows read 'depends on'. Fixing $\delta_{\text{IPL}} = 0$ puts every θ_i on the IPL scale.

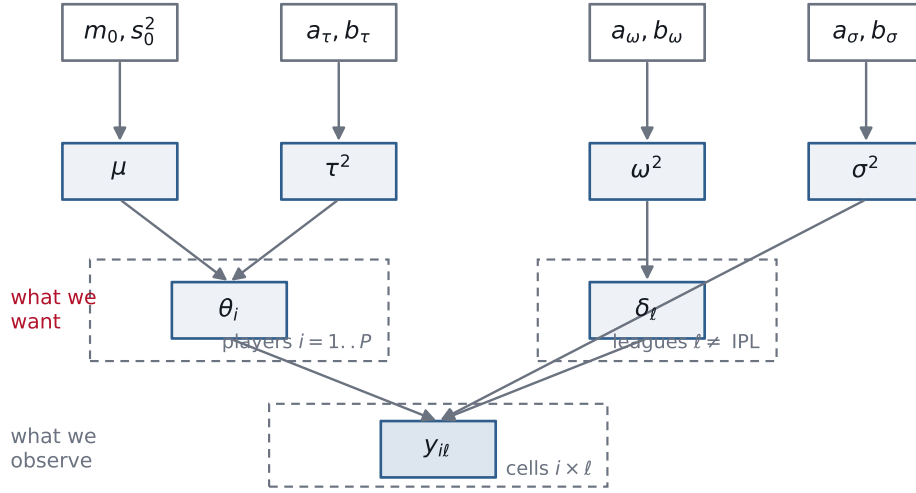


Figure 2. M3 as a directed acyclic graph. Arrows read “depends on”. The plates mark the three groups of quantities that repeat: batters, leagues, and the cells we observe.

4 Prior specification

The priors are weakly informative. Each one is chosen so that it can be justified in a sentence.

Parameter	Prior	Where the number comes from
μ	$N(130, 20^2)$	T20 batting averages a strike rate near 130; ± 40 covers anything sane.
τ^2	$\text{IG}(3, 450)$	Prior mean 225, i.e. $\tau \approx 15$: most batters within ± 30 of the mean.
ω^2	$\text{IG}(3, 128)$	Prior mean 64, i.e. $\omega \approx 8$: leagues differ by single-digit strike-rate points.
σ^2	$\text{IG}(3, 52,000)$	Prior mean 26,000, from the ball-by-ball run distribution: $\text{Var}(X) = 2.80$ runs per ball $\Rightarrow \sigma^2 \approx 27,978$.

Table 3. Priors, and where each number comes from.

The prior for σ^2 is the one with a direct empirical basis. Runs off the bat are distributed as follows across the fitting window:

Runs off the bat	0	1	2	3	4	5	6
Share of balls	0.379	0.378	0.069	0.004	0.112	0.000	0.057

giving $E[X] = 1.320$, $\text{Var}(X) = 2.798$ and hence $\sigma^2 \approx 27,978$ on the strike-rate scale. We centre the prior there. Section 8 reports what the data do to that number.

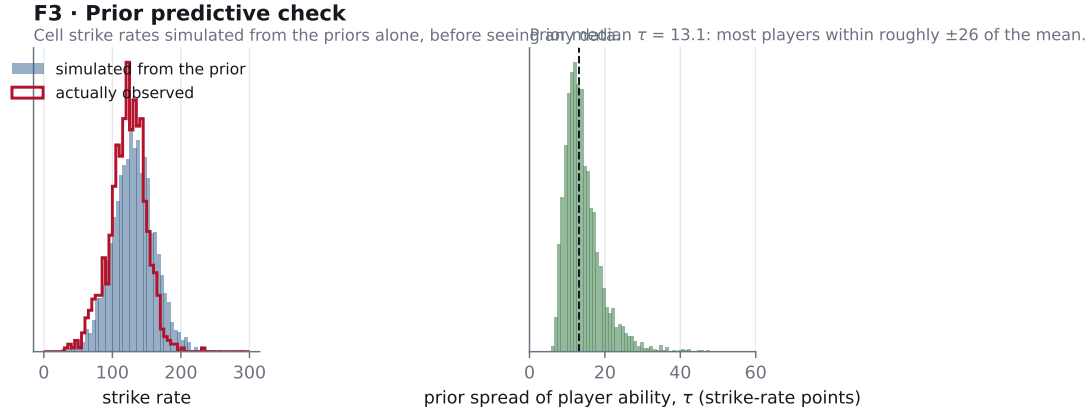


Figure 3. Prior predictive check. Left: cell strike rates simulated from the priors alone, against the strike rates we actually observe. The prior is wider than reality, which is what a weakly informative prior should be, and it does not put serious mass on absurdities. Right: the implied prior spread of batting ability.

5 The Gibbs sampler

All six full conditionals are available in closed form, so the sampler is a pure Gibbs sampler (Sessions 8–9). No Metropolis step is required. The model is additive, and an additive Normal model with conjugate priors remains conjugate throughout. A log link, which is the natural alternative for a rate, would have removed conjugacy for θ and required Metropolis-within-Gibbs. This is one reason the model is specified on the additive scale.

Write cells $c = 1, \dots, C$ with batter $p(c)$, league $\ell(c)$, balls n_c , strike rate y_c ; P batters and L leagues.

5.1 The six full conditionals

1. Batting ability. With $r_c = y_c - \delta_{\ell(c)}$, the terms in the joint posterior that involve θ_i are the cells belonging to batter i and the population prior:

$$p(\theta_i | -) \propto \exp \left\{ -\frac{1}{2\sigma^2} \sum_{c: p(c)=i} n_c (r_c - \theta_i)^2 \right\} \exp \left\{ -\frac{(\theta_i - \mu)^2}{2\tau^2} \right\},$$

which is Normal by completing the square:

$$\theta_i | - \sim N \left(\frac{\sum_c n_c r_c / \sigma^2 + \mu / \tau^2}{V_i^{-1}}, V_i \right), \quad V_i^{-1} = \sum_{c: p(c)=i} \frac{n_c}{\sigma^2} + \frac{1}{\tau^2}.$$

The posterior precision is the precision from the batter's own record plus the precision from the population. This is PS4 Q3(e) and PS5 Q2(f).

A special case worth noting. With one league and one cell, $V_i^{-1} = n_i / \sigma^2 + 1 / \tau^2$ and the mean collapses to

$$E(\theta_i | -) = w_i \bar{y}_i + (1 - w_i) \mu, \quad w_i = \frac{n_i \tau^2}{\sigma^2 + n_i \tau^2},$$

which is PS5 Q1(b). Here w_i is the share of the estimate that comes from the batter's own record and $1 - w_i$ the share that comes from the population. It is the key quantity in the model, and the interactive companion provides a slider for it.

2. League offsets, for $\ell \neq \text{IPL}$, with $s_c = y_c - \theta_{p(c)}$, by the same completion of the square over the cells played in league ℓ :

$$\delta_\ell | - \sim N \left(W_\ell \sum_{c: \ell(c)=\ell} \frac{n_c s_c}{\sigma^2}, W_\ell \right), \quad W_\ell^{-1} = \sum_{c: \ell(c)=\ell} \frac{n_c}{\sigma^2} + \frac{1}{\omega^2}.$$

δ_{IPL} is never sampled; it is held at zero.

3–6. The remaining four. Each follows the same pattern: a Normal for a location and an inverse-gamma for a variance, with the data contributing a sum of squares and the prior contributing its own shape and scale:

$$\begin{aligned}\mu \mid - &\sim N\left(\frac{\sum_i \theta_i / \tau^2 + m_0 / s_0^2}{P / \tau^2 + 1 / s_0^2}, \left(\frac{P}{\tau^2} + \frac{1}{s_0^2}\right)^{-1}\right), & \tau^2 \mid - &\sim \text{IG}\left(a_\tau + \frac{P}{2}, b_\tau + \frac{1}{2} \sum_i (\theta_i - \mu)^2\right), \\ \omega^2 \mid - &\sim \text{IG}\left(a_\omega + \frac{L-1}{2}, b_\omega + \frac{1}{2} \sum_{\ell \neq \text{IPL}} \delta_\ell^2\right), & \sigma^2 \mid - &\sim \text{IG}\left(a_\sigma + \frac{C}{2}, b_\sigma + \frac{1}{2} \sum_c n_c e_c^2\right),\end{aligned}$$

with $e_c = y_c - \theta_{p(c)} - \delta_{\ell(c)}$. The weight n_c appears in the last one because the likelihood variance is σ^2 / n_c , not σ^2 .

Each is a componentwise update in the sense of PS4 Q8: we sample one block at a time given the rest. The full derivations are in Appendix A.

5.2 Settings

Four chains, 12,000 iterations each, 4,000 discarded as burn-in, thinned by 4, giving 8,000 draws. The θ update is vectorised over cells using a single grouped sum, so a full run takes a few seconds.

6 MCMC diagnostics

There are two separate questions here. Is the code correct, and have the chains converged?

6.1 Four checks on the code

The sandbox used for this project has no R installation, so we cannot cross-check against `rstanarm` or `brms`. The sampler is therefore tested directly.

1. **One batter, one league, against the closed form.** With μ , τ^2 and σ^2 pinned, the posterior must be the Normal–Normal answer from Sessions 3–4. The closed-form mean agrees with the $w\bar{y} + (1-w)\mu$ form to 10^{-14} (that part is algebra, not simulation), and the sampler’s mean agrees with it to within 0.09 of a Monte Carlo standard error.
2. **Recovery from simulated data.** Generating from known θ , δ , σ^2 , τ^2 and ω^2 and refitting recovers every one inside its 95% credible interval, with 94.3% of the true θ_i inside theirs.
3. **Limiting cases (PS5 Q1(e)).** As τ^2 falls through 400, 100, 25, 4, 1 the posterior spread of θ collapses monotonically from 13.2 to 0.06 and lands on the complete-pooling value; as $\tau^2 \rightarrow \infty$ the posterior means reproduce the raw no-pooling means to within 3.0 Monte Carlo standard errors. One point is worth recording. A centred Gibbs sampler run at $\tau^2 = 10^{-6}$ does *not* reach the complete-pooling answer in finite time. It stalls, because at that setting almost no information flows from the data to μ . This is the mixing problem discussed in PS4 Q8(e) and Q13(e).
4. **Two independent implementations.** The interactive companion runs the same six conditionals written from scratch in JavaScript, with a completely different random number generator (mulberry32 with Box–Muller, against NumPy’s PCG64). On the same data the two agree: $\mu = 132.19$ in Python against 132.20 in JavaScript, no parameter differing by more than 0.97 Monte Carlo standard errors, and the 600 posterior mean abilities correlating at 0.9994 with an RMSE of 0.24 strike-rate points. Two independent implementations agreeing this closely is strong evidence that both are correct.

6.2 Convergence

Parameter	Mean	SD	2.5%	97.5%	$\hat{R}$	ESS
μ	132.19	1.40	129.38	134.90	1.0015	1,161
τ^2	131.02	19.24	95.80	171.07	1.0005	1,761
ω^2	53.94	33.79	19.12	137.88	1.0005	7,073
σ^2	53,870.91	3,603.31	47,329.13	61,352.99	1.0006	3,597
δ_{CPL}	-7.90	1.95	-11.68	-4.03	1.0012	2,453
δ_{BBL}	-4.97	1.80	-8.49	-1.49	1.0012	2,112
δ_{T2OI}	-5.25	1.45	-8.16	-2.41	1.0013	1,766
all 600 θ_i (worst)					1.0016	4,772

Table 4. Posterior summaries and convergence diagnostics for M3. $\hat{R}$ is the split version; ESS is the Geyer initial-positive-sequence estimator. The last row is the worst case across all 600 batter abilities.

The worst $\hat{R}$ anywhere in the model, including every one of the 600 individual abilities, is 1.0016. The smallest effective sample size is 5,263, against 8,000 stored draws; the median is 7,133.

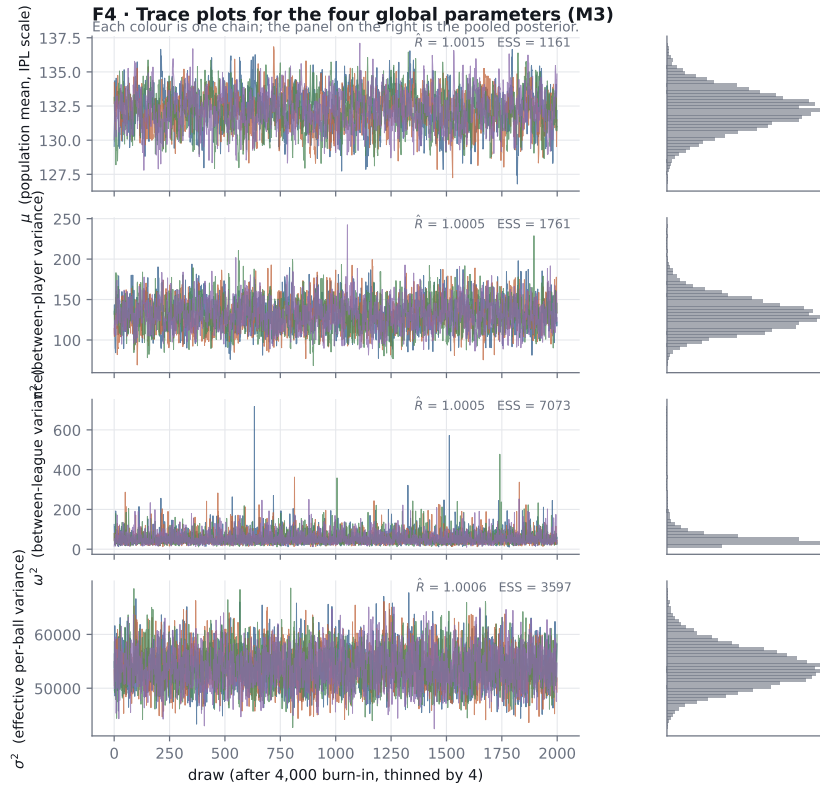


Figure 4. Four chains for the four global parameters, started from different values. The right panel of each row is the pooled posterior. No chain drifts and none becomes stuck.

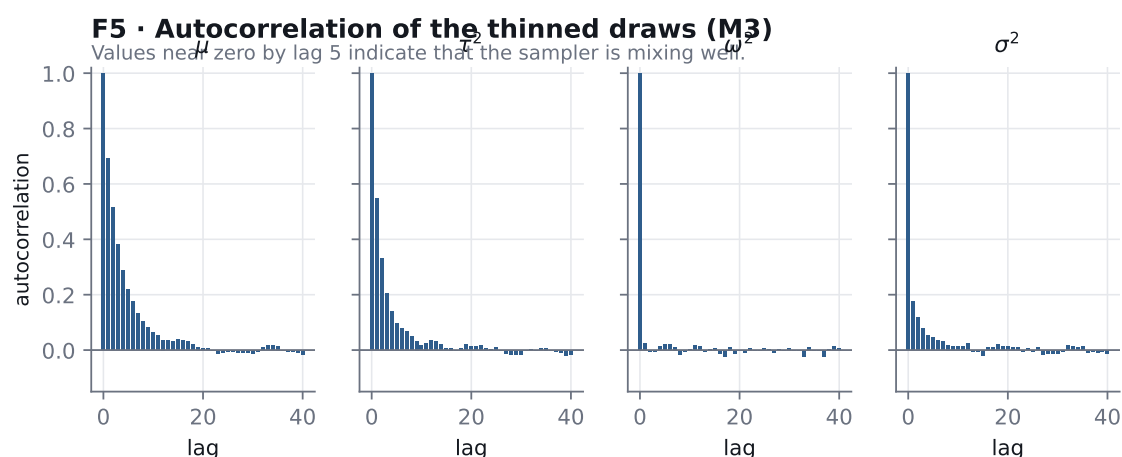


Figure 5. Autocorrelation of the thinned draws. Near zero by lag 5 for every global parameter, which is why the effective sample sizes are close to the number of draws.

7 Posterior predictive checks

Convergence tells us that the sampler explored the posterior. It says nothing about whether the model describes the data. For that we use the posterior predictive check of *Bayes Rules!* Chapter 10: simulate the data the fitted model would generate and compare it with the data observed. Following PS4 Q11(d), the replicated cells include fresh Normal errors and not only the fitted means.

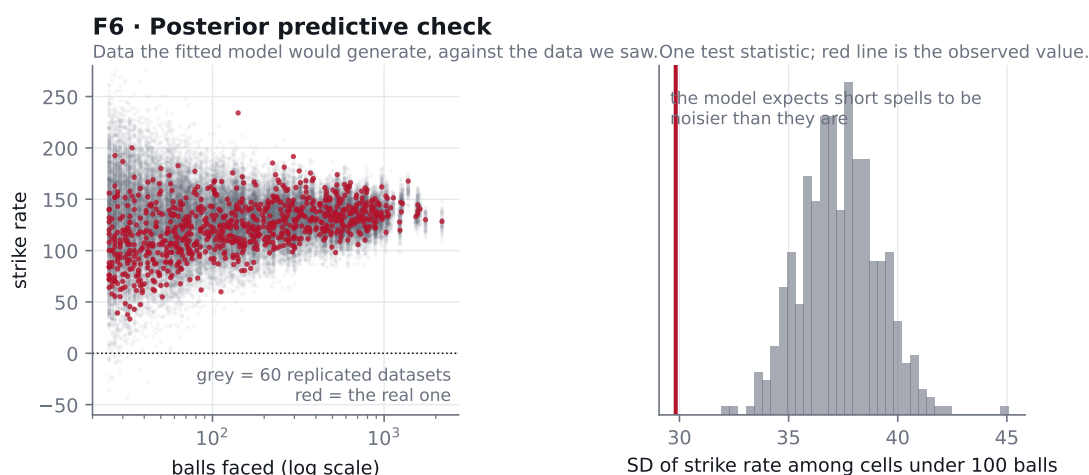


Figure 6. Left: 60 replicated datasets in grey against the real one in red. Right: one test statistic — the spread of strike rates among cells with fewer than 100 balls.

What the model reproduces. The ball-weighted run rate of each of the four competitions is reproduced almost exactly. For the IPL it is 139.5 observed against 139.1 replicated, a posterior predictive p -value of 0.37, and the other three are equally unremarkable. The highest strike rate in the data is comfortably inside what the model expects. Crucially, after restricting T20Is to Full Member matches, the replicated data contain essentially no impossible values — 0.04% of replicated cells fall below zero, against 0.3% before the restriction. The Normal likelihood survives its most obvious objection.

Where it does not fit. Two related failures, both in the same direction.

- The model expects short spells to be noisier than they are. Among cells under 100 balls the observed spread of strike rates is 29.8 against a replicated 37.4 ($p = 1.00$).
- It produces too many very high cells: 19.4% of replicated cells strike above 150, against

12.8% observed.

Both follow from using a single σ^2 for every cell size. The next section explains why that single number is pulled upward.

8 Results

8.1 The residual variance, and what it says about independence

The fitted residual variance is $\sigma^2 = 53,871$. The variance implied by counting runs off the bat is 27,978. The ratio is **1.93**.

The gap is not an error. It is informative. If balls were independent given ability, there would be nowhere in the model for form, matchups, pitch conditions or batting position to go. Instead all of that variation enters σ^2 , which is therefore an *effective* per-ball variance rather than a pure sampling variance. PS5 Q1(g) gives the reasoning: once outcomes within a group are correlated, the effective sample size is smaller than the nominal one. Here 100 balls carry roughly the information of 52.

Two estimates of the same quantity. Counting runs gives ± 16.7 points for a 100-ball strike rate. The fitted model, which also absorbs the variation of a batter between spells, gives ± 23.2 . Neither figure is the one an auction currently uses.

This also explains the misfit found in Section 7. A single σ^2/n is pulled upward by the long cells, where the extra correlated variation matters most, and then over-predicts the noise in short ones. Section 11 gives the conjugate correction and explains why we did not adopt it.

8.2 The population

$\mu = 132.2$, $\tau = 11.4$ and $\omega = 7.3$. In words: the average batter across the four competitions would strike at about 132.2 in the IPL, about two-thirds of batters lie within 11.4 points of that, and the competitions themselves differ by single-digit strike-rate points.

8.3 Shrinkage and the leaderboard

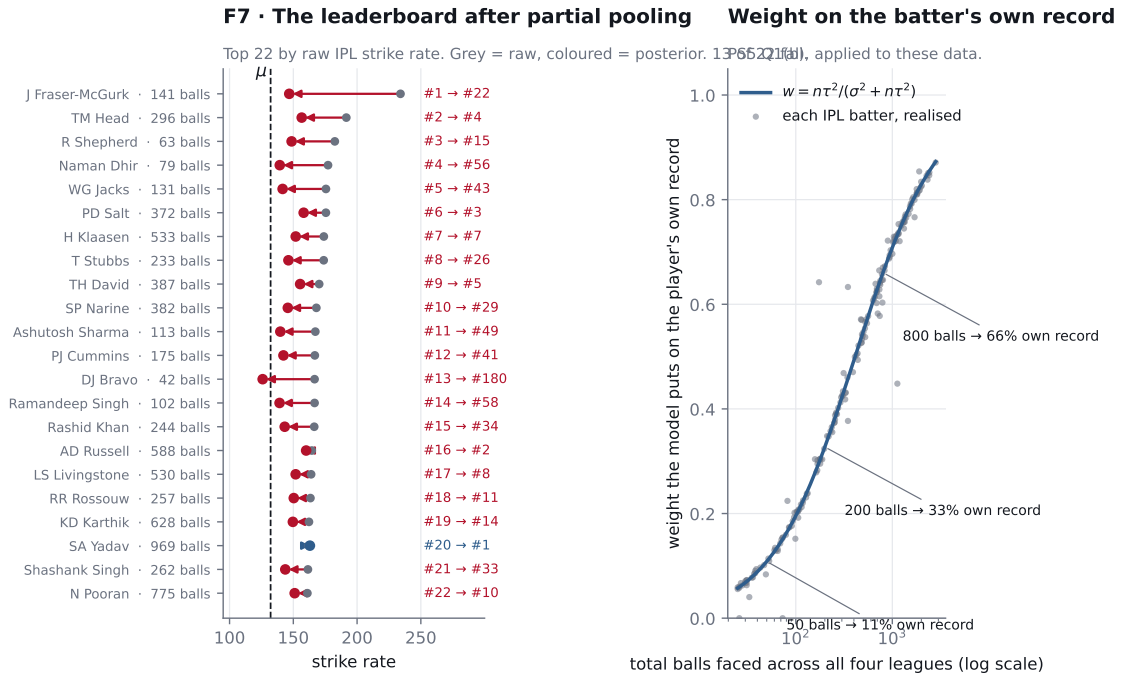


Figure 7. Left: the 22 batters at the top of the raw IPL leaderboard, with an arrow from raw strike rate to posterior mean and the rank change beside each. Right: the weight the model puts on a batter's own record, against the theoretical curve $w = n\tau^2 / (\sigma^2 + n\tau^2)$.

13 of the top 22 move down. The clearest case is DJ Bravo: a strike rate of 167 off 42 IPL balls, ranked 13 on the raw list and 180 once the number of balls is taken into account. The clearest move upward is SA Yadav, ranked 20 on the raw list and 1 afterwards, because his record is long enough to be taken close to face value.

The right-hand panel compares the shrinkage each batter actually experienced, computed from the posterior, with the theoretical weight $w = n\tau^2 / (\sigma^2 + n\tau^2)$. The two agree closely. The model is applying PS5 Q1(b) to 600 batters.

8.4 League offsets

F8 · What a strike rate is worth in each league

Posterior for δ_ℓ : how much a league's numbers must be adjusted to read as IPL numbers.

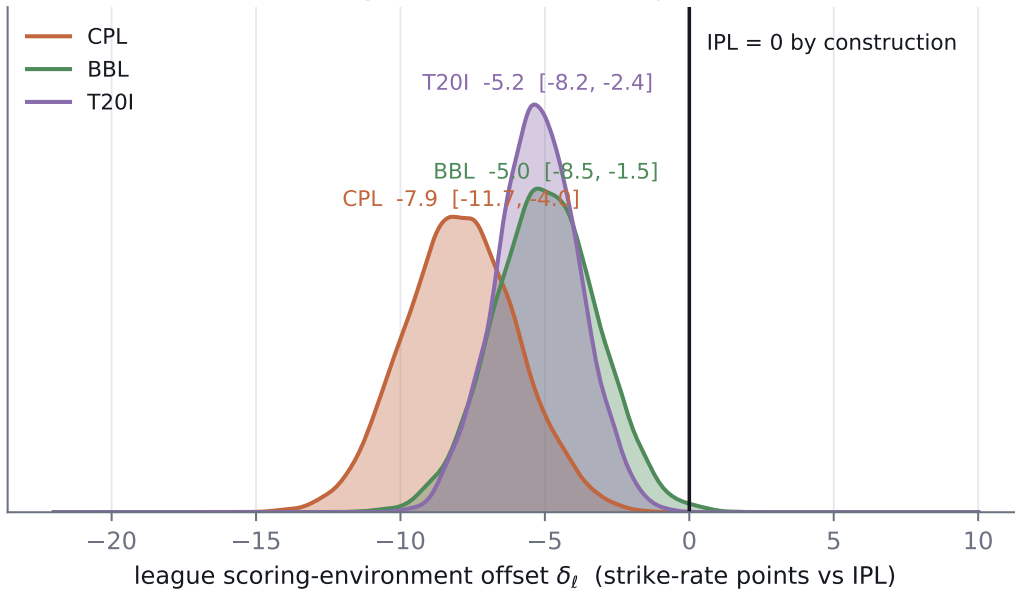


Figure 8. Posterior distributions of the league offsets, against the IPL fixed at zero.

League	δ_ℓ	95% credible interval
CPL	-7.9	(-11.7, -4.0)
BBL	-5.0	(-8.5, -1.5)
T20I	-5.2	(-8.2, -2.4)

All three offsets are below the IPL and all three intervals exclude zero. The IPL is the highest-scoring of the four environments: a strike rate of 140 in the CPL reads as about $140 - 7.9 \approx 132$ on the IPL scale. The ordering, with the IPL above the BBL and elite T20Is and the CPL lowest, matches the published CricViz ranking of T20 standards, which uses the same bridge-player reasoning. We add credible intervals, which that ranking does not report. The CPL and BBL intervals are wide for the reason given in Section 2: only 38 and 36 batters respectively link those competitions to the IPL directly.

We describe δ_ℓ as a *scoring-environment offset* rather than a quality adjustment. It combines bowling standard, pitches, boundary sizes and format. The decision requires the translation between scales; it does not require knowing which of these is responsible.

9 Model comparison and predictive accuracy

9.1 Predictive accuracy using WAIC and LOO

Model		$\widehat{\text{elpd}}_{\text{WAIC}}$	p_{WAIC}	ΔWAIC	$\widehat{\text{elpd}}_{\text{LOO}}$	LOOIC	cells with $\hat{k} > 0$
M0	complete pooling	-900.1	2.1	166.7	-900.1	1800.3	0 / 1
M1	no pooling	-816.8	117.0	0.0	-887.0	1774.1	188 / 1
M2	partial pooling, IPL only	-894.6	39.7	155.7	-899.8	1799.5	9 / 1
M3	partial pooling + league offsets	-878.4	45.3	123.3	-884.0	1768.0	9 / 1

Table 5. WAIC and PSIS-LOO for the four models, all evaluated on the same 194 IPL cells so that the numbers are comparable. $\hat{k}$ is the Pareto shape from PSIS; values above 0.7 mean the estimate for that cell is unreliable.

Taken at face value, WAIC prefers M1, the raw leaderboard. The diagnostics show why this reading should not be accepted. M1 has one cell per batter and one parameter per batter, so the model is saturated: its effective number of parameters is 117.0 out of 194 observations. PSIS-LOO reports $\hat{k} > 0.7$ for 188 of the 194 cells, which means the importance sampling that leave-one-out relies on has broken down almost everywhere. The reason is easy to state: removing a batter's only cell removes everything the model knew about that batter.

Among the models for which the diagnostic is well behaved, M3 has the highest LOO estimate (-884.0, against -899.8 for M2 and -900.1 for M0). The conclusion we draw from this table is limited: an in-sample criterion cannot settle a comparison in which one model is saturated. The 2025 holdout can, and the next section reports it.

9.2 Out-of-sample validation

We fit on 2021–2024 and predict each batter's 2025 IPL strike rate. The results are stratified by how much IPL history the batter had *before* 2025, because that is the quantity the argument turns on. **The structure of the table below was fixed before it was run.**

Prior IPL exposure	<i>n</i>	M0	M1	M2	M3
RMSE (bootstrap SE)					
No IPL record, has overseas data	8	39.6 (16.2)	<i>undefined</i>	39.4 (15.8)	33.7 (11.6)
No record in any of the four leagues	10	44.4 (5.7)	<i>undefined</i>	45.2 (5.7)	49.5 (6.2)
1–100 IPL balls	6	22.8 (5.3)	44.4 (14.3)	21.9 (5.1)	22.6 (5.8)
101–300 IPL balls	21	29.7 (5.7)	39.0 (9.6)	28.8 (5.3)	26.4 (4.1)
300+ IPL balls	58	26.0 (2.1)	23.6 (2.1)	24.5 (2.1)	24.7 (2.1)
All with an IPL record	85	26.7 (2.2)	29.9 (3.6)	25.5 (2.0)	25.0 (1.8)
All 103 holdout batters	103	30.1 (2.5)	29.9 (3.7)	29.2 (2.4)	29.0 (2.2)
<i>Coverage of the 95% posterior predictive interval, all 103 batters</i>					
Nominal 95%	103	95%	79%	95%	93%

Table 6. Root mean squared error in predicting 2025 IPL strike rate, with bootstrap standard errors, and coverage of the 95% posterior predictive interval. Best in each row in bold.

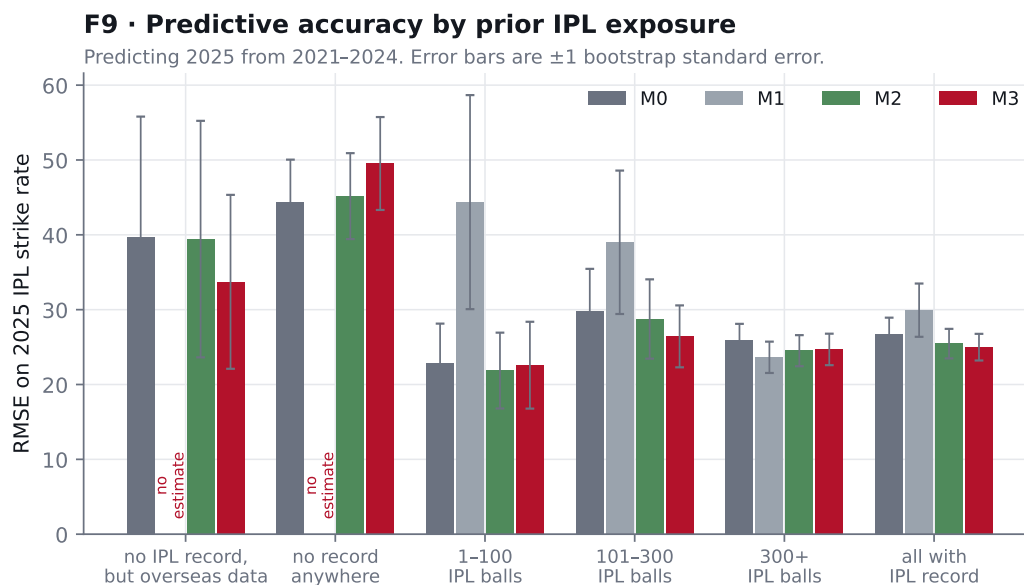


Figure 9. The same comparison as a picture. M1 has no bar in the first two groups because it has no estimate to give.

Reading the table

Batters with no IPL record but an overseas record. 8 batters played in the 2025 IPL with no prior IPL record but a record in the CPL, BBL or T20Is. For them M3 has an RMSE of 33.7, against 39.4 for IPL-only partial pooling and 39.6 for complete pooling, while **M1 gives no estimate at all**. For exactly the batters a franchise most needs to price, the conventional method does not simply perform worse; it is unavailable. This is the second case in PS5 Q5(i), a new student at an existing university: the hierarchy has a source of information for such a batter, so it produces a usable answer.

Batters with some IPL record. Across all 85 of them the four models order as expected: M3 25.0, M2 25.5, M0 26.7, M1 29.9. In the 101–300 ball group, where partial pooling has the most to contribute, M3 has an RMSE of 26.4 against 39.0 for the raw leaderboard.

Batters with long IPL careers. M1 23.6, M3 24.7, M2 24.5. These are within sampling error of one another, which is what the model predicts: at 300 balls and above, w exceeds 0.8 and partial pooling has little left to add. We report this rather than claim an improvement that is not there.

Calibration. The 95% posterior predictive intervals cover 93%, 95% and 95% of the actual 2025 strike rates. The no-pooling intervals cover 79%. A franchise using raw strike rates is therefore overconfident as well as less accurate. Part of the cause is structural: M1 is saturated, so it cannot estimate its own σ^2 — the posterior for σ^2 behaves close to a random walk — and has little information left with which to size its own uncertainty.

The one group where the prediction went the other way

10 batters in the 2025 IPL had no record in *any* of our four competitions. For them M3 is worse: 49.5 against 44.4 for the IPL-only models.

We report this because the cause is instructive. For a batter with no data, every model returns its prior, so this row compares priors rather than models. The population under M3 is all 600 batters across four competitions and its μ is 132.2; the IPL-only models see only IPL regulars and sit near 138. A 2025 IPL debutant is not a random draw from world T20 cricket, since somebody selected him, and the narrower prior happens to be better calibrated for this particular group. This is PS1 Q3(c), the limits of a population-level prior, and it is the selection-bias limitation appearing directly in the results.

The difference concerns priors for eighteen batters, not the likelihood, and it does not affect any group for which data exist. It also points to the natural extension: a population model that conditions on which competitions a batter has been selected for.

10 The decision rule and prior sensitivity

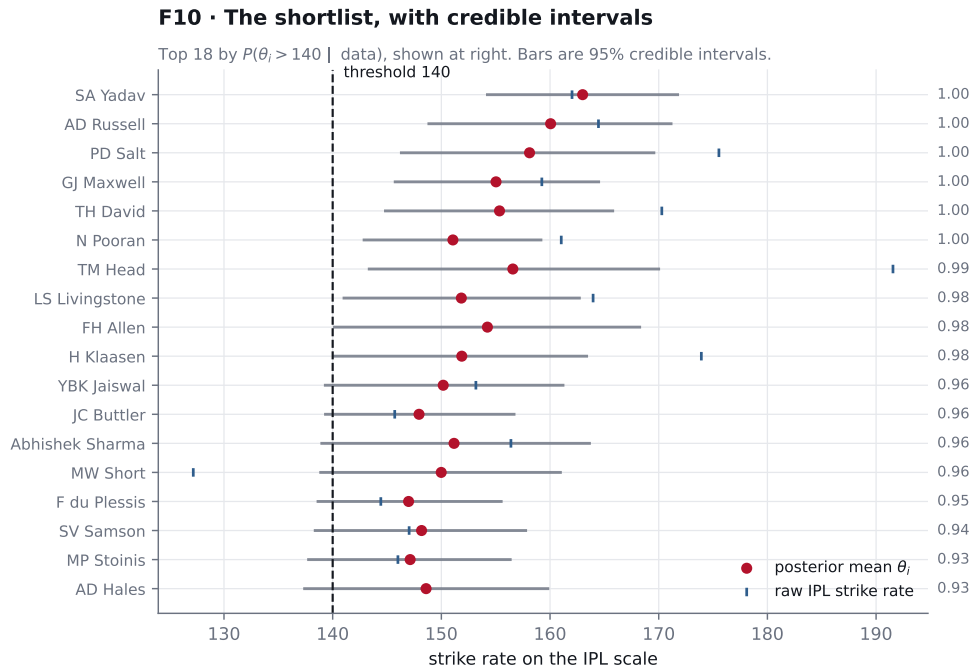


Figure 10. The shortlist at $c = 140$, $p = 0.80$, restricted to batters with at least 150 balls across the four competitions. Bars are 95% credible intervals; the tick is the raw IPL strike rate where one exists; the number on the right is $\Pr(\theta_i > 140 | \text{data})$.

At the baseline setting, 28 batters clear the threshold. The top of the list — SA Yadav, AD Russell, GJ Maxwell, TH David, PD Salt, N Pooran — contains no surprises, which is a useful check: a method that produced an unrecognisable list would not be trusted. The differences appear further down. DJ Bravo is eleventh on the raw list at 166.7 and has $\Pr(\theta > 140) = 0.05$. Batters with strong overseas records and no IPL history appear on the model’s list and cannot appear on the raw one at all.

10.1 Prior sensitivity

Following PS1 Q1(b)–(c) and PS4 Q12(e), we do not vary the prior arbitrarily. We use the priors of three people who would plausibly disagree about the same decision.

Stakeholder	Prior on μ	Prior on τ^2	$E[\mu y]$	$E[\tau y]$	Shortlist	Top-10 shared
Scout	$N(140, 25^2)$	$IG(3, 800)$	132.0	11.8	31	10 / 10
Analyst	$N(130, 20^2)$	$IG(3, 450)$	132.2	11.4	28	10 / 10
CFO	$N(130, 10^2)$	$IG(6, 250)$	132.6	10.8	26	10 / 10
Stress	$N(130, 5^2)$	$IG(40, 250)$	137.8	2.9	1	8 / 10

Table 7. Prior sensitivity. The Scout believes ability really does vary; the CFO has watched franchises overpay for one good season. “Stress” is not a plausible belief — it is a deliberate extreme in which almost all apparent difference between T20 batters is noise.

The three plausible priors agree. They give an identical top ten, shortlists of 31, 28 and 26 batters, and Jaccard overlap between 0.90 and 0.93. Following PS4 Q12(f), this insensitivity indicates that the data are informative: with 839 cells, and long careers for the batters who matter, the likelihood dominates the prior.

The stress test shows where the conclusion would break. Setting $\tau \approx 2.9$, which corresponds to the belief that T20 batters are nearly interchangeable, leaves 8 of the top 10 names in place but reduces the shortlist from 28 batters to 1. The *ranking* is robust to the prior; a decision based

on an *absolute threshold* is not, because under that prior no batter can be shown to exceed 140. A franchise that wants a ranking need not worry much about the prior. A franchise that wants a yes or no answer at a fixed strike rate does.

11 Limitations

1. **The Normal likelihood.** Strike rates are bounded below by zero and skewed right; the Normal is neither. We model a *mean* of at least 25 balls, so the central limit theorem applies, and the posterior predictive check finds essentially no impossible replicated values. It does show that the model over-states the noise in short spells.
2. **Selection.** We observe only batters who were selected. Absolute ability estimates are therefore biased upward. The *comparison* between methods is unaffected, because every method is scored on the same batters, but Section 9.2 shows the selection appearing directly in the population prior for batters with no record at all.
3. **Ability is treated as fixed across five years.** In reality batters improve, decline and change roles. We restrict the window to 2021–2025 to limit this, and part of what inflates σ^2 is genuine change over time being absorbed as noise.
4. **δ_ℓ combines several effects.** Bowling standard, pitches, boundary sizes and format all enter a single number, which is why we call it a scoring-environment offset. The T20I restriction in Section 2 illustrates what happens when a “league” is not a single environment.
5. **Balls are not conditionally independent.** PS1 Q11(j) lists the reasons — defensive pressure, situation, fatigue, matchups. The fitted σ^2 is 1.93 times the independence value, which is the model measuring exactly this. The clean fix stays conjugate: give each cell its own latent level, $y_c \sim N(\eta_c, \sigma^2/n_c)$ with $\eta_c \sim N(\theta_{p(c)} + \delta_{\ell(c)}, \kappa^2)$. This adds a seventh full conditional, itself Normal, and separates sampling noise from variation between spells. We did not adopt it, because the model specification was fixed at the outset, and because the diagnosis is more useful here than the extra parameter.
6. **Strike rate is not the whole of batting.** A batter who strikes at 160 and is dismissed every other over is worth less than one who strikes at 145 and bats through. T20 value has at least two dimensions and we have modelled one. The same hierarchy with a Beta–Binomial layer for dismissals is the natural extension.
7. **The offsets are only as good as the bridge.** 106 batters connect the IPL and elite T20Is, but only 38 and 36 connect the CPL and BBL directly. Those two offsets depend partly on the T20I route, which is reflected in the width of their credible intervals.

12 Conclusion

The raw leaderboard is not an arbitrary method. It is the estimator obtained by assuming that a batter’s own record contains everything relevant about him and that other competitions contain nothing, which is what no pooling assumes. Stated in that form, the assumption is difficult to defend.

Replacing it costs one hierarchical layer and one offset per league. The gains are specific: better predictions for batters with short IPL records, predictions that exist at all for batters with none, intervals that cover 95% of outcomes rather than 79%, and a shortlist with a probability attached to every name.

Two numbers bracket the problem. A strike rate over 100 balls carries ± 16.7 points of sampling noise, and the fitted model, which also absorbs the variation of a batter between spells, puts the figure at ± 23.2 . A decision taken on a raw strike rate uses neither.

Everything in this report is reproducible from the repository, and the sampler runs in the browser so that a reader can change our priors and see the effect.

A Derivations of the six full conditionals

The joint posterior, up to a constant, is

$$\begin{aligned}
p(\theta, \delta, \mu, \tau^2, \omega^2, \sigma^2 \mid y) &\propto \prod_{c=1}^C (\sigma^2/n_c)^{-1/2} \exp\left\{-\frac{n_c(y_c - \theta_{p(c)} - \delta_{\ell(c)})^2}{2\sigma^2}\right\} \\
&\times \prod_{i=1}^P (\tau^2)^{-1/2} \exp\left\{-\frac{(\theta_i - \mu)^2}{2\tau^2}\right\} \prod_{\ell \neq \text{IPL}} (\omega^2)^{-1/2} \exp\left\{-\frac{\delta_\ell^2}{2\omega^2}\right\} \\
&\times \exp\left\{-\frac{(\mu - m_0)^2}{2s_0^2}\right\} (\tau^2)^{-(a_\tau+1)} e^{-b_\tau/\tau^2} (\omega^2)^{-(a_\omega+1)} e^{-b_\omega/\omega^2} (\sigma^2)^{-(a_\sigma+1)} e^{-b_\sigma/\sigma^2}.
\end{aligned}$$

Each full conditional keeps only the factors containing the parameter in question.

θ_i . Collect the exponent terms in θ_i :

$$-\frac{1}{2} \left[\theta_i^2 \left(\sum_{c:p(c)=i} \frac{n_c}{\sigma^2} + \frac{1}{\tau^2} \right) - 2\theta_i \left(\sum_{c:p(c)=i} \frac{n_c r_c}{\sigma^2} + \frac{\mu}{\tau^2} \right) \right] + \text{const},$$

with $r_c = y_c - \delta_{\ell(c)}$. A quadratic in θ_i with these coefficients is the kernel of $N(m, V)$ with V^{-1} the coefficient of θ_i^2 and $m = V \times$ the coefficient of θ_i , giving conditional 1. With $L = 1$ and one cell this is $m = w\bar{y} + (1-w)\mu$ with $w = n\tau^2/(\sigma^2 + n\tau^2)$.

δ_ℓ . Identical algebra with $s_c = y_c - \theta_{p(c)}$ and prior mean 0, summing over cells in league ℓ . δ_{IPL} is fixed at 0 and has no conditional.

μ . Only the P ability priors and μ 's own prior involve μ ; the same completion of the square gives precision $P/\tau^2 + 1/s_0^2$.

$\tau^2, \omega^2, \sigma^2$. Each appears as $(\cdot)^{-(a+1+k/2)} \exp\{-(b + \frac{1}{2}\text{SS})/\cdot\}$, which is the inverse-gamma kernel with shape $a + k/2$ and scale $b + \frac{1}{2}\text{SS}$: $k = P$ and $\text{SS} = \sum (\theta_i - \mu)^2$ for τ^2 ; $k = L - 1$ and $\text{SS} = \sum \delta_\ell^2$ for ω^2 ; $k = C$ and $\text{SS} = \sum n_c(y_c - \theta_{p(c)} - \delta_{\ell(c)})^2$ for σ^2 . The weight n_c appears because the likelihood variance is σ^2/n_c , not σ^2 .

B Where each piece of the course is used

What we use	Where it comes from
Prior, posterior, posterior predictive	Session 2
Normal–Normal conjugacy, precision-weighted posterior mean	Sessions 3–4; <i>Bayes Rules!</i> 5.1–5.3
Inverse-gamma prior for a variance	Sessions 3–5
Monte Carlo estimation of a posterior probability	Session 7; PS4 Q9(f)
Gibbs sampling and full conditionals	Sessions 8–9
Componentwise updating; centring and mixing	PS4 Q8(c)–(e), Q13(e)
Convergence diagnostics, $\hat{R}$, autocorrelation	Sessions 10–11; <i>Bayes Rules!</i> 7.1–7.6
Posterior prediction from MCMC output; fresh errors	Session 12; PS4 Q10, Q11(d)
Normal means and Bayesian regression machinery	Sessions 13–14
Hierarchical models, partial pooling, DAGs, plates	Sessions 17–18; <i>Bayes Rules!</i> 16.1–16.5
Shrinkage weight $w = n\tau^2 / (\sigma^2 + n\tau^2)$	PS5 Q1(b)
Complete / no / partial pooling and limiting cases	PS5 Q1(e)–(f)
Within-group correlation and effective sample size	PS5 Q1(g)
Prediction for an existing unit, a new unit in a known group, and a new group	PS5 Q5(i)
Posterior predictive checking and cross-validation	<i>Bayes Rules!</i> Ch. 10
Separating inference from the decision rule	PS1 Q9(h)–(i), Q11(i)
Prior sensitivity as competing stakeholders	PS1 Q1(b)–(c); PS4 Q12(e)–(f)
Conditional independence and where it fails	PS1 Q11(j)
Limits of a population-level prior for a brand-new unit	PS1 Q3(b)–(c)

C The code

Every number in this report is produced by the scripts below; nothing is typed by hand. The full repository, including the interactive site, is laid out as follows.

Path	What it does
prep/build_data.py	Cricsheet archives $\rightarrow$ cells.json, holdout.json, ballruns.json. Assembles the T20I aggregate file, drops wides, applies the Full Member filter and the 25-ball floor, and prints the bridge matrix.
analysis/gibbs.py	The six full conditionals and the M0–M3 ladder.
analysis/rng.py	Seed derivation; master seed 20260901.
analysis/dataio.py	Loads the cells; full and re-indexed ipl views.
analysis/diagnostics.py	Split $\hat{R}$, Geyer ESS, autocorrelation.
analysis/waic_loo.py	WAIC and PSIS-LOO with generalised Pareto tail fitting.
analysis/tests_correctness.py	Correctness tests 1–3.
analysis/test4_js_vs_pytho	Test 4: drives the browser sampler headlessly.
analysis/run_main.py	Fits M0–M3, writes draws and the diagnostics table.
analysis/validation.py	The 2025 holdout study, five buckets, bootstrap errors.
analysis/compare.py	WAIC / LOO on the common 194 IPL cells.
analysis/ppc.py	Posterior predictive checks.
analysis/sensitivity.py	Scout / Analyst / CFO / stress priors.
analysis/figures.py, style.py	Figures F1–F10.
analysis/tables.py	Tables T1–T6 as L ^A T _E X fragments.
analysis/report_numbers.py	Every quoted number, as a L ^A T _E X macro.
analysis/export_app.py	Writes the JSON the site reads.
app/	The interactive site: no libraries, no build step.

C.1 prep/build_data.py

```

1 """
2 Beyond Strike Rate --data preparation.
3
4 Reads the four Cricsheet CSV (csv2) archives, restricts to 2021–2025, aggregates
5 ball-by-ball deliveries to player x league cells, and writes:
6
7     data/cells.json training cells (2021–2024, all four leagues)
8     data/holdout.json test cells (2025, IPL only)
9     data/ballruns.json empirical distribution of runs off the bat (for sigma^2)
10
11 Also prints the league x league bridge-player overlap matrix, which is the
12 identification check for the league-offset parameters delta[l].
13
14 Cricsheet ships an 'all_matches.csv' inside each csv2 zip; we use it directly.
15 Unzip each archive into raw/<league>/ and run: python3 prep/build_data.py
16 """
17 import pandas as pd, numpy as np, json, os, glob
18
19 LEAGUES = {"ipl": 0, "cpl": 1, "bbl": 2, "t20i": 3}
20 YEARS = range(2021, 2026)
21 MIN_BALLS = 25
22
23 USECOLS = ["match_id", "start_date", "striker", "runs_off_bat", "wides",
24            "batting_team", "bowling_team"]
25
26 # T20 internationals are played by 105 different teams, from India and Australia
27 # down to Estonia and Mongolia. That is not one scoring environment, so a single
28 # league offset cannot describe it. We keep only matches in which BOTH sides are
29 # ICC Full Members, which is the T20I most people mean.
30 FULL_MEMBERS = {"India", "Australia", "England", "South Africa", "New Zealand",
31                 "Pakistan", "Sri Lanka", "Bangladesh", "West Indies",
32                 "Afghanistan", "Ireland", "Zimbabwe"}
33
34 frames = []

```

```

35 def ensure_all_matches(name):
36     """The t20s_male_csv2 archive ships per-match files only; build the
37     aggregate file once by concatenating them (headers are identical)."""
38     path = f"raw/{name}/all_matches.csv"
39     if os.path.exists(path):
40         return path
41     files = sorted(f for f in glob.glob(f"raw/{name}/*.csv")
42                   if not f.endswith("_info.csv"))
43     with open(path, "w", newline="") as out:
44         for k, f in enumerate(files):
45             with open(f) as fh:
46                 header = fh.readline()
47                 if k == 0:
48                     out.write(header)
49                 out.writelines(fh)
50     return path
51
52 for name, idx in LEAGUES.items():
53     d = pd.read_csv(ensure_all_matches(name), usecols=USECOLS, low_memory=False)
54     d["league"] = idx
55     frames.append(d)
56     print(f"{name:5s} raw deliveries: {len(d):>9,}")
57
58 df = pd.concat(frames, ignore_index=True)
59 df["year"] = pd.to_datetime(df["start_date"], errors="coerce").dt.year
60 df = df[df["year"].isin(YEARS)]
61
62 # A wide is NOT a ball faced by the batter.
63 df = df[df["wides"].isna()]
64 print(f"\nDeliveries 2021-2025, wides excluded: {len(df):,}")
65
66 before = (df["league"] == LEAGUES["t20i"]).sum()
67 keep = (df["league"] != LEAGUES["t20i"]) | (df["batting_team"].isin(FULL_MEMBERS)
68                                     & df["bowling_team"].isin(FULL_MEMBERS))
69 df = df[keep]
70 after = (df["league"] == LEAGUES["t20i"]).sum()
71 print(f"T20I deliveries kept (Full Member v Full Member only): {after:,} of {before:,}")
72
73 def aggregate(frame):
74     g = (frame.groupby(["striker", "league"])
75          .agg(n=("runs_off_bat", "size"), runs=("runs_off_bat", "sum"))
76          .reset_index())
77     g = g[g["n"] >= MIN_BALLS]
78     g["y"] = 100.0 * g["runs"] / g["n"]
79     return g
80
81 train = aggregate(df[df["year"] <= 2024])
82 test = aggregate(df[(df["year"] == 2025) & (df["league"] == LEAGUES["ipl"])])
83
84 players = sorted(set(train["striker"]) | set(test["striker"]))
85 pidx = {p: i for i, p in enumerate(players)}
86
87 os.makedirs("data", exist_ok=True)
88
89 json.dump({
90     "meta": {"window": "2021-2024", "minBalls": MIN_BALLS,
91             "leagues": list(LEAGUES), "iplIndex": 0},
92     "players": players,
93     "cells": [{"p": pidx[r.striker], "l": int(r.league),
94               "n": int(r.n), "y": round(float(r.y), 3)}
95               for r in train.itertuples()],
96 }, open("data/cells.json", "w"))
97
98 json.dump({
99     "meta": {"season": 2025, "league": "ipl"},
100    "cells": [{"p": pidx[r.striker], "n": int(r.n), "y": round(float(r.y), 3)}
101              for r in test.itertuples()],
102 }, open("data/holdout.json", "w"))
103
104 # Empirical per-ball run distribution (training window) -> sigma^2 for the prior.
105 tr = df[df["year"] <= 2024]
106 vc = tr["runs_off_bat"].value_counts().sort_index()
107 json.dump({"values": [int(v) for v in vc.index], "counts": [int(c) for c in vc.values]},
108           open("data/ballruns.json", "w"))

```

```

109
110 x = tr["runs_off_bat"].to_numpy(dtype=float)
111 m1, m2 = x.mean(), (x**2).mean()
112 var = m2 - m1**2
113 print(f"\nPer-ball runs off the bat: E[X]={m1:.4f} E[X^2]={m2:.4f} Var={var:.4f}")
114 print(f"sigma^2 on the x100 strike-rate scale = {var*1e4:,.0f} sigma = {np.sqrt(var)*100:.1f}")
115 print(f"SD of observed strike rate off 100 balls = {np.sqrt(var)*100/10:.1f} SR points")
116
117 # --- go / no-go: bridge-player overlap matrix ---
118 piv = train.pivot_table(index="striker", columns="league",
119                          values="n", aggfunc="sum").notna().astype(int)
120 print("\nLeague x League overlap (players appearing in both):")
121 print(piv.T.values @ piv.values)
122 print("\nLeague order:", list(LEAGUES))
123 print(f"\nPlayers: {len(players)} Train cells: {len(train)} Test cells: {len(test)}")
124 print("Cells per league:", train.groupby('league').size().to_dict())

```

C.2 analysis/gibbs.py — the sampler

```

1  """
2  Gibbs samplers for the model ladder M0-M3.
3
4  Cells c = 1..C with player p[c], league l[c], balls n[c], strike rate y[c].
5
6      M3 y[c] ~ N( theta[p[c]] + delta[l[c]], sigma2 / n[c] )
7      theta[l] ~ N(mu, tau2) delta[l] ~ N(0, omega2), delta[IPL] = 0
8      mu ~ N(m0, s0^2) tau2 ~ IG(a_tau, b_tau)
9      omega2 ~ IG(a_om, b_om) sigma2 ~ IG(a_sig, b_sig)
10
11 All six full conditionals are closed form; there is no Metropolis step.
12 M2 is M3 restricted to IPL cells with delta identically zero.
13 M1 is no pooling (flat prior on each theta), M0 complete pooling (one theta).
14
15 Everything is vectorised over cells with np.bincount, so a 12,000-iteration
16 chain over 839 cells runs in a couple of seconds.
17 """
18 import numpy as np
19 from dataclasses import dataclass, field
20
21 IPL = 0
22
23 # ----- priors -----
24 @dataclass
25 class Priors:
26     m0: float = 130.0 # mu ~ N(m0, s0^2)
27     s0: float = 20.0
28     a_tau: float = 3.0 # tau2 ~ IG(a_tau, b_tau) E[tau2] = b/(a-1) = 225 -> sd 15
29     b_tau: float = 450.0
30     a_om: float = 3.0 # omega2 ~ IG(a_om, b_om) E[omega2] = 64 -> sd 8
31     b_om: float = 128.0
32     a_sig: float = 3.0 # sigma2 ~ IG(a_sig, b_sig) E[sigma2] = 26000
33     b_sig: float = 52000.0
34
35
36 @dataclass
37 class Data:
38     p: np.ndarray # player index per cell
39     l: np.ndarray # league index per cell
40     n: np.ndarray # balls faced
41     y: np.ndarray # strike rate
42     P: int # number of players
43     L: int # number of leagues
44
45     @property
46     def C(self) -> int:
47         return len(self.y)
48
49
50 def inv_gamma(rng, a, b, size=None):
51     """X ~ IG(a, b) <=> 1/X ~ Gamma(shape=a, rate=b)."""
52     return 1.0 / rng.gamma(shape=a, scale=1.0 / b, size=size)
53

```

```

54
55 # ----- sampler -----
56 def gibbs(data: Data, priors: Priors, model: str = "M3",
57         iters: int = 12000, burn: int = 4000, thin: int = 4,
58         rng: np.random.Generator = None,
59         tau2_fixed: float = None, sigma2_fixed: float = None,
60         keep_theta: bool = True):
61     """Run one chain. Returns a dict of draws (post burn-in, thinned)."""
62     rng = rng or np.random.default_rng(0)
63     P, L, C = data.P, data.L, data.C
64     p, l, n, y = data.p, data.l, data.n, data.y
65
66     n_by_player = np.bincount(p, weights=n, minlength=P)
67     has_data = n_by_player > 0
68
69     # --- initialise -----
70     ybar = np.divide(np.bincount(p, weights=n * y, minlength=P), n_by_player,
71                     out=np.full(P, float(priors.m0)), where=has_data)
72     theta = np.where(has_data, ybar, priors.m0).astype(float)
73     delta = np.zeros(L)
74     mu = float(np.mean(theta[has_data])) if has_data.any() else priors.m0
75     tau2 = 225.0 if tau2_fixed is None else tau2_fixed
76     omega2 = 64.0
77     sigma2 = 25000.0 if sigma2_fixed is None else sigma2_fixed
78
79     if model == "M0": # complete pooling: one shared theta
80         theta = np.full(P, mu)
81
82     keep = np.arange(burn, iters, thin)
83     K = len(keep)
84     out = {k: np.empty(K) for k in ("mu", "tau2", "omega2", "sigma2")}
85     out["theta"] = np.empty((K, P)) if keep_theta else None
86     out["delta"] = np.empty((K, L))
87     out["loglik"] = np.empty((K, C))
88     k = 0
89
90     for it in range(iters):
91         # ---- 1. theta -----
92         if model == "M0":
93             # single theta: precision-weighted over all cells + prior on mu
94             prec = n.sum() / sigma2 + 1.0 / priors.s0**2
95             mean = (np.sum(n * (y - delta[l])) / sigma2 + priors.m0 / priors.s0**2) / prec
96             th0 = mean + rng.normal() / np.sqrt(prec)
97             theta[:] = th0
98             mu = th0
99         else:
100             r = y - delta[l]
101             Sn = np.bincount(p, weights=n, minlength=P) / sigma2
102             Sr = np.bincount(p, weights=n * r, minlength=P) / sigma2
103             if model == "M1": # no pooling: flat prior on theta
104                 prec = np.where(Sn > 0, Sn, 1.0)
105                 mean = np.where(Sn > 0, Sr / prec, priors.m0)
106                 theta = mean + rng.normal(size=P) / np.sqrt(prec)
107             else: # M2 / M3: hierarchical
108                 prec = Sn + 1.0 / tau2
109                 mean = (Sr + mu / tau2) / prec
110                 theta = mean + rng.normal(size=P) / np.sqrt(prec)
111
112         # ---- 2. delta (M3 only; delta[IPL] pinned to 0) -----
113         if model == "M3":
114             s = y - theta[p]
115             Sn_l = np.bincount(l, weights=n, minlength=L) / sigma2
116             Ss_l = np.bincount(l, weights=n * s, minlength=L) / sigma2
117             for j in range(L):
118                 if j == IPL:
119                     delta[j] = 0.0
120                     continue
121                 prec = Sn_l[j] + 1.0 / omega2
122                 delta[j] = Ss_l[j] / prec + rng.normal() / np.sqrt(prec)
123
124         # ---- 3. mu -----
125         if model in ("M2", "M3"):
126             prec = P / tau2 + 1.0 / priors.s0**2
127             mean = (theta.sum() / tau2 + priors.m0 / priors.s0**2) / prec

```

```

128     mu = mean + rng.normal() / np.sqrt(prec)
129
130     # ---- 4. tau2 -----
131     if model in ("M2", "M3") and tau2_fixed is None:
132         tau2 = inv_gamma(rng, priors.a_tau + P / 2.0,
133                          priors.b_tau + 0.5 * np.sum((theta - mu) ** 2))
134
135     # ---- 5. omega2 -----
136     if model == "M3":
137         free = np.array([j for j in range(L) if j != IPL])
138         omega2 = inv_gamma(rng, priors.a_om + len(free) / 2.0,
139                           priors.b_om + 0.5 * np.sum(delta[free] ** 2))
140
141     # ---- 6. sigma2 -----
142     if sigma2_fixed is None:
143         resid = y - theta[p] - delta[l]
144         sigma2 = inv_gamma(rng, priors.a_sig + C / 2.0,
145                             priors.b_sig + 0.5 * np.sum(n * resid ** 2))
146
147     # ---- store -----
148     if it >= burn and (it - burn) % thin == 0 and k < K:
149         out["mu"][k] = mu
150         out["tau2"][k] = tau2
151         out["omega2"][k] = omega2
152         out["sigma2"][k] = sigma2
153         out["delta"][k] = delta
154         if keep_theta:
155             out["theta"][k] = theta
156         resid = y - theta[p] - delta[l]
157         out["loglik"][k] = (-0.5 * np.log(2 * np.pi * sigma2 / n)
158                             - n * resid ** 2 / (2 * sigma2))
159         k += 1
160
161     return out
162
163
164 def run_chains(data, priors, model="M3", chains=4, iters=12000, burn=4000,
165               thin=4, study="main", tau2_fixed=None, sigma2_fixed=None):
166     """Run 'chains' independent chains and stack the draws."""
167     from rng import chain_rng
168     res = [gibbs(data, priors, model, iters, burn, thin,
169                  chain_rng(model, c, study), tau2_fixed, sigma2_fixed)
170            for c in range(chains)]
171     stacked = {}
172     for key in ("mu", "tau2", "omega2", "sigma2"):
173         stacked[key] = np.stack([r[key] for r in res]) # (chains, K)
174     for key in ("theta", "delta", "loglik"):
175         if res[0][key] is not None:
176             stacked[key] = np.stack([r[key] for r in res]) # (chains,K,.)
177     return stacked

```

C.3 analysis/diagnostics.py

```

1  """Split R-hat, effective sample size, autocorrelation --Gelman et al. (BDA3 §11.4)."""
2  import numpy as np
3
4  def split_rhat(x):
5      """x: (chains, draws). Split each chain in half, then the usual R-hat."""
6      x = np.asarray(x, dtype=float)
7      m, n = x.shape
8      n2 = n // 2
9      s = np.concatenate([x[:, :n2], x[:, n2:2 * n2]], axis=0) # (2m, n2)
10     M, N = s.shape
11     means = s.mean(axis=1)
12     varis = s.var(axis=1, ddof=1)
13     W = varis.mean()
14     B = N * means.var(ddof=1)
15     if W <= 0:
16         return np.nan
17     var_hat = (N - 1) / N * W + B / N
18     return float(np.sqrt(var_hat / W))
19
20 def _acf(v, maxlag):

```



```

21     v = v - v.mean()
22     d = np.correlate(v, v, mode="full")[len(v) - 1:]
23     d = d / d[0] if d[0] != 0 else d
24     return d[:maxlag + 1]
25
26 def acf(x, maxlag=40):
27     """Mean autocorrelation across chains."""
28     x = np.asarray(x, dtype=float)
29     return np.mean([_acf(c, maxlag) for c in x], axis=0)
30
31 def ess(x):
32     """Effective sample size, Geyer initial-positive-sequence estimator."""
33     x = np.asarray(x, dtype=float)
34     m, n = x.shape
35     maxlag = min(n - 2, 500)
36     rho = np.mean([_acf(c, maxlag) for c in x], axis=0)
37     # pair the autocorrelations and stop at the first non-positive pair sum
38     s = 0.0
39     for t in range(1, maxlag - 1, 2):
40         pair = rho[t] + rho[t + 1]
41         if pair <= 0:
42             break
43         s += pair
44     return float(m * n / (1.0 + 2.0 * s))
45
46 def summarise(draws, name):
47     """R-hat / ESS / mean / sd / 95% CI for a scalar parameter."""
48     x = np.asarray(draws, dtype=float)
49     flat = x.ravel()
50     return dict(param=name, mean=flat.mean(), sd=flat.std(ddof=1),
51                lo=np.percentile(flat, 2.5), hi=np.percentile(flat, 97.5),
52                rhat=split_rhat(x), ess=ess(x))

```

C.4 app/js/gibbs.js — the same sampler, independently written

```

1  /* The same six full conditionals as analysis/gibbs.py, in the browser.
2
3  y[c] ~ N( theta[p[c]] + delta[l[c]], sigma2 / n[c] )
4  theta[i] ~ N(mu, tau2) delta[l] ~ N(0, omega2), delta[IPL] = 0
5  mu ~ N(m0, s0^2) tau2 ~ IG(a_tau, b_tau) omega2 ~ IG(a_om, b_om) sigma2 ~ IG(a_sig, b_sig)
6
7  Models: M0 complete pooling, M1 no pooling, M2 partial pooling (IPL only),
8  M3 partial pooling with league offsets. */
9  (function (root) {
10     'use strict';
11     const IPL = 0;
12
13     const DEFAULT_PRIORS = {
14         m0: 130, s0: 20, a_tau: 3, b_tau: 450,
15         a_om: 3, b_om: 128, a_sig: 3, b_sig: 52000
16     };
17
18     function runGibbs(opts) {
19         const data = opts.data; // {p,l,n,y,P,L}
20         const pr = Object.assign({}, DEFAULT_PRIORS, opts.priors || {});
21         const model = opts.model || 'M3';
22         const iters = opts.iters || 6000,
23               burn = opts.burn || 2000,
24               thin = opts.thin || 2;
25         const rng = new root.RNG(opts.seed === undefined ? 1 : opts.seed);
26         const onProgress = opts.onProgress;
27         const tau2Fixed = opts.tau2Fixed;
28
29         const P = data.P, L = data.L, C = data.n.length;
30         const p = data.p, l = data.l, n = data.n, y = data.y;
31
32         const nByPlayer = new Float64Array(P), sumByPlayer = new Float64Array(P);
33         for (let c = 0; c < C; c++) { nByPlayer[p[c]] += n[c]; sumByPlayer[p[c]] += n[c] * y[c];
34             }
35
36         const theta = new Float64Array(P);
37         for (let i = 0; i < P; i++) theta[i] = nByPlayer[i] > 0 ? sumByPlayer[i] / nByPlayer[i] :
38             pr.m0;

```

```

37 const delta = new Float64Array(L);
38 let mu = pr.m0, tau2 = tau2Fixed !== undefined && tau2Fixed !== null ? tau2Fixed : 225,
39     omega2 = 64, sigma2 = 25000;
40 if (model === 'M0') { mu = 0; for (let i = 0; i < P; i++) mu += theta[i] / P; }
41
42 const nKeep = Math.max(1, Math.floor((iters - burn) / thin));
43 const keep = { mu: [], tau2: [], omega2: [], sigma2: [],
44               delta: [], [], [], [], theta: null, iters: [] };
45 const thetaSum = new Float64Array(P), thetaSq = new Float64Array(P);
46 const watch = opts.watch === undefined ? -1 : opts.watch;
47 const watchDraws = [];
48 let kept = 0;
49
50 const Sn = new Float64Array(P), Sr = new Float64Array(P);
51 const SnL = new Float64Array(L), SsL = new Float64Array(L);
52
53 for (let it = 0; it < iters; it++) {
54   /* 1. theta */
55   if (model === 'M0') {
56     let tot = 0, num = 0;
57     for (let c = 0; c < C; c++) { tot += n[c]; num += n[c] * (y[c] - delta[l[c]]); }
58     const prec = tot / sigma2 + 1 / (pr.s0 * pr.s0);
59     const mean = (num / sigma2 + pr.m0 / (pr.s0 * pr.s0)) / prec;
60     const th0 = mean + rng.randn() / Math.sqrt(prec);
61     theta.fill(th0); mu = th0;
62   } else {
63     Sn.fill(0); Sr.fill(0);
64     for (let c = 0; c < C; c++) {
65       Sn[p[c]] += n[c] / sigma2;
66       Sr[p[c]] += n[c] * (y[c] - delta[l[c]]) / sigma2;
67     }
68     if (model === 'M1') {
69       for (let i = 0; i < P; i++) {
70         if (Sn[i] > 0) theta[i] = Sr[i] / Sn[i] + rng.randn() / Math.sqrt(Sn[i]);
71         else theta[i] = pr.m0 + pr.s0 * rng.randn();
72       }
73     } else {
74       for (let i = 0; i < P; i++) {
75         const prec = Sn[i] + 1 / tau2;
76         theta[i] = (Sr[i] + mu / tau2) / prec + rng.randn() / Math.sqrt(prec);
77       }
78     }
79   }
80
81   /* 2. delta (M3 only, delta[IPL] pinned to zero) */
82   if (model === 'M3') {
83     SnL.fill(0); SsL.fill(0);
84     for (let c = 0; c < C; c++) {
85       SnL[l[c]] += n[c] / sigma2;
86       SsL[l[c]] += n[c] * (y[c] - theta[p[c]]) / sigma2;
87     }
88     for (let j = 0; j < L; j++) {
89       if (j === IPL) { delta[j] = 0; continue; }
90       const prec = SnL[j] + 1 / omega2;
91       delta[j] = SsL[j] / prec + rng.randn() / Math.sqrt(prec);
92     }
93   }
94
95   /* 3. mu */
96   if (model === 'M2' || model === 'M3') {
97     const prec = P / tau2 + 1 / (pr.s0 * pr.s0);
98     let s = 0; for (let i = 0; i < P; i++) s += theta[i];
99     const mean = (s / tau2 + pr.m0 / (pr.s0 * pr.s0)) / prec;
100    mu = mean + rng.randn() / Math.sqrt(prec);
101  }
102
103  /* 4. tau2 */
104  if ((model === 'M2' || model === 'M3') && (tau2Fixed === undefined || tau2Fixed ===
105      null)) {
106    let ss = 0; for (let i = 0; i < P; i++) { const d = theta[i] - mu; ss += d * d; }
107    tau2 = rng.randInvGamma(pr.a_tau + P / 2, pr.b_tau + 0.5 * ss);
108  }
109  /* 5. omega2 */

```

```

110     if (model === 'M3') {
111         let ss = 0, k = 0;
112         for (let j = 0; j < L; j++) if (j !== IPL) { ss += delta[j] * delta[j]; k++; }
113         omega2 = rng.randInvGamma(pr.a_om + k / 2, pr.b_om + 0.5 * ss);
114     }
115
116     /* 6. sigma2 */
117     let ss = 0;
118     for (let c = 0; c < C; c++) {
119         const r = y[c] - theta[p[c]] - delta[l[c]];
120         ss += n[c] * r * r;
121     }
122     sigma2 = rng.randInvGamma(pr.a_sig + C / 2, pr.b_sig + 0.5 * ss);
123
124     /* store */
125     if (it >= burn && (it - burn) % thin === 0) {
126         keep.mu.push(mu); keep.tau2.push(tau2);
127         keep.omega2.push(omega2); keep.sigma2.push(sigma2);
128         for (let j = 0; j < L; j++) keep.delta[j].push(delta[j]);
129         keep.iters.push(it);
130         for (let i = 0; i < P; i++) { thetaSum[i] += theta[i]; thetaSq[i] += theta[i] * theta[
131             i]; }
132         if (watch >= 0) watchDraws.push(theta[watch]);
133         kept++;
134     }
135     if (onProgress && (it % 250 === 0)) onProgress(it / iters);
136 }
137
138 const mean = new Float64Array(P), sd = new Float64Array(P);
139 for (let i = 0; i < P; i++) {
140     mean[i] = thetaSum[i] / kept;
141     sd[i] = Math.sqrt(Math.max(0, thetaSq[i] / kept - mean[i] * mean[i]));
142 }
143 keep.thetaMean = mean; keep.thetaSd = sd; keep.watchDraws = watchDraws;
144 keep.nKept = kept;
145 return keep;
146 }
147
148 /* ---- diagnostics, same definitions as analysis/diagnostics.py ---- */
149 function splitRhat(chains) {
150     const halves = [];
151     chains.forEach(c => {
152         const h = Math.floor(c.length / 2);
153         halves.push(c.slice(0, h)); halves.push(c.slice(h, 2 * h));
154     });
155     const M = halves.length, N = halves[0].length;
156     if (N < 4) return NaN;
157     const means = halves.map(h => h.reduce((a, b) => a + b, 0) / N);
158     const vars = halves.map((h, k) =>
159         h.reduce((a, b) => a + (b - means[k]) * (b - means[k]), 0) / (N - 1));
160     const W = vars.reduce((a, b) => a + b, 0) / M;
161     const gm = means.reduce((a, b) => a + b, 0) / M;
162     const B = N * means.reduce((a, b) => a + (b - gm) * (b - gm), 0) / (M - 1);
163     if (W <= 0) return NaN;
164     return Math.sqrt(((N - 1) / N * W + B / N) / W);
165 }
166
167 function acf(x, maxlag) {
168     const n = x.length, m = x.reduce((a, b) => a + b, 0) / n;
169     const d = x.map(v => v - m);
170     let c0 = 0; for (let i = 0; i < n; i++) c0 += d[i] * d[i];
171     const out = [];
172     for (let k = 0; k <= maxlag; k++) {
173         let s = 0; for (let i = 0; i < n - k; i++) s += d[i] * d[i + k];
174         out.push(c0 === 0 ? 0 : s / c0);
175     }
176     return out;
177 }
178
179 function ess(chains) {
180     const m = chains.length, n = chains[0].length;
181     const maxlag = Math.min(n - 2, 200);
182     const rho = new Array(maxlag + 1).fill(0);

```

```

182   chains.forEach(c => { const a = acf(c, maxlag); for (let k = 0; k <= maxlag; k++) rho[k]
      += a[k] / m; });
183   let s = 0;
184   for (let t = 1; t < maxlag - 1; t += 2) {
185     const pair = rho[t] + rho[t + 1];
186     if (pair <= 0) break;
187     s += pair;
188   }
189   return m * n / (1 + 2 * s);
190 }
191
192 root.Gibbs = { run: runGibbs, splitRhat, acf, ess, DEFAULT_PRIORS, IPL };
193 })(typeof self !== 'undefined' ? self : this);

```

C.5 app/js/rng.js — Box-Muller and Marsaglia-Tsang in the browser

```

1  /* Random number generation for the browser.
2   randn : Box-Muller (Sessions 8-9 notes)
3   randGamma : Marsaglia-Tsang (2000)
4   randInvGamma :  $X \sim \text{Gamma}(a, \text{rate } b) \Rightarrow 1/X \sim \text{InvGamma}(a, b)$ 
5   A seeded generator (mulberry32) is used so a run in the browser is reproducible. */
6  (function (root) {
7    'use strict';
8
9    function mulberry32(seed) {
10     let a = seed >>> 0;
11     return function () {
12       a |= 0; a = (a + 0x6D2B79F5) | 0;
13       let t = Math.imul(a ^ (a >>> 15), 1 | a);
14       t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
15       return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
16     };
17   }
18
19   function RNG(seed) {
20     this.u = mulberry32(seed === undefined ? 20260901 : seed);
21     this.spare = null;
22   }
23
24   RNG.prototype.unif = function () {
25     let u = this.u();
26     return u === 0 ? 1e-12 : u;
27   };
28
29   RNG.prototype.randn = function () {
30     if (this.spare !== null) { const s = this.spare; this.spare = null; return s; }
31     const u = this.unif(), v = this.unif();
32     const r = Math.sqrt(-2 * Math.log(u));
33     this.spare = r * Math.sin(2 * Math.PI * v);
34     return r * Math.cos(2 * Math.PI * v);
35   };
36
37   /* Gamma with shape a and RATE b, so the mean is a/b. */
38   RNG.prototype.randGamma = function (a, b) {
39     if (a < 1) return this.randGamma(a + 1, b) * Math.pow(this.unif(), 1 / a);
40     const d = a - 1 / 3, c = 1 / Math.sqrt(9 * d);
41     for (;;) {
42       let x, v;
43       do { x = this.randn(); v = 1 + c * x; } while (v <= 0);
44       v = v * v * v;
45       const u = this.unif();
46       if (u < 1 - 0.0331 * x * x * x * x) return d * v / b;
47       if (Math.log(u) < 0.5 * x * x + d * (1 - v + Math.log(v))) return d * v / b;
48     }
49   };
50
51   RNG.prototype.randInvGamma = function (a, b) { return 1 / this.randGamma(a, b); };
52
53   root.RNG = RNG;
54 })(typeof self !== 'undefined' ? self : this);

```